

WebGL

Tvorba internetových aplikací

6. 3. 2013

Michal Ferko

Demo time

- <http://cycleblob.com/>
- http://www.youtube.com/watch?v=AJg_BmY9-8o
- <http://hexgl.bkcore.com/>
- <http://chrome.angrybirds.com/>

WebGL overview

- Open standard for **hardware-accelerated** display of 3D scenes in a web browser
- Derived from OpenGL ES 2.0
- JavaScript-based
- Displays content into a HTML5 `<canvas>`

OpenGL

- Open Graphics Library
- Open standard for GPU accelerated rendering since 1991
- Current version 4.3
- Large amount of vendor/system-specific extensions
- Main rival: Direct3D
- Direct3D not for Unix-based systems

OpenGL applications

- Games
 - Deus Ex, Counter-strike, Doom 3, Far Cry, Half-life, Left 4 Dead, Quake, StarCraft II, Star Wars KOTOR, Team Fortress, WoW, ...
- Other
 - Google Earth, Blender, 3ds max, SketchUp, Photoshop, ...

OpenGL ES

- OpenGL for Embedded Systems
- Subset of OpenGL API for “weaker” hardware
- For mobile and other platforms
 - Android, BlackBerry, Nokia Symbian, ...
 - Apple products - iPad, iPhone, ...
 - Google Native Client
- WebGL is essentially OpenGL ES 2.0 for web browsers

OpenGL ES 2.0

- Removed fixed-function pipeline
 - Everything has to be rendered with shaders
 - Need your own matrix functions
 - No more lighting and materials, need to do it on your own
- Not backward compatible with OpenGL ES 1.1

WebGL

- OpenGL ES 2.0 API in JavaScript
- Small differences because of JavaScript
- **No need for plugins** (except IE)
- Supported by new versions of all major browsers, even mobile ones
- Ultimate cross-platform tool for graphics development
- Speed is a huge downside

WebGL development

- Not one line of code without
 - OpenGL ES 2.0 Reference Pages
 - <http://www.khronos.org/opengles/sdk/docs/man/>
 - WebGL Quick Reference Card
 - http://www.khronos.org/files/webgl/webgl-reference-card-1_0.pdf
 - Alternatively the WebGL Cheat Sheet
 - http://www.nihilogic.dk/labs/webgl_cheat_sheet/WebGL_Cheat_Sheet.pdf
 - Khronos WebGL wiki
 - http://www.khronos.org/webgl/wiki/Main_Page
 - Khronos WebGL specification
 - <http://www.khronos.org/registry/webgl/specs/latest/>

Getting started

- Get supported browser
 - Opera requires explicit enabling of WebGL
- Check support with some of the demos
 - <http://get.webgl.org/> - official WebGL browser support test
 - <http://www.chromeexperiments.com/webgl/>

Useful tutorials

- <http://learningwebgl.com/>
- <http://www.khronos.org/webgl/wiki/Tutorial>
- <http://dev.opera.com/articles/view/raw-webgl-part1-getting-started/>
- <http://net.tutsplus.com/tutorials/webgl-essentials-part-i/>
- <http://namchok.com/webgl/>

Preparing canvas

HTML

```
<canvas id="app" width="640" height="480">  
</canvas>
```

JS

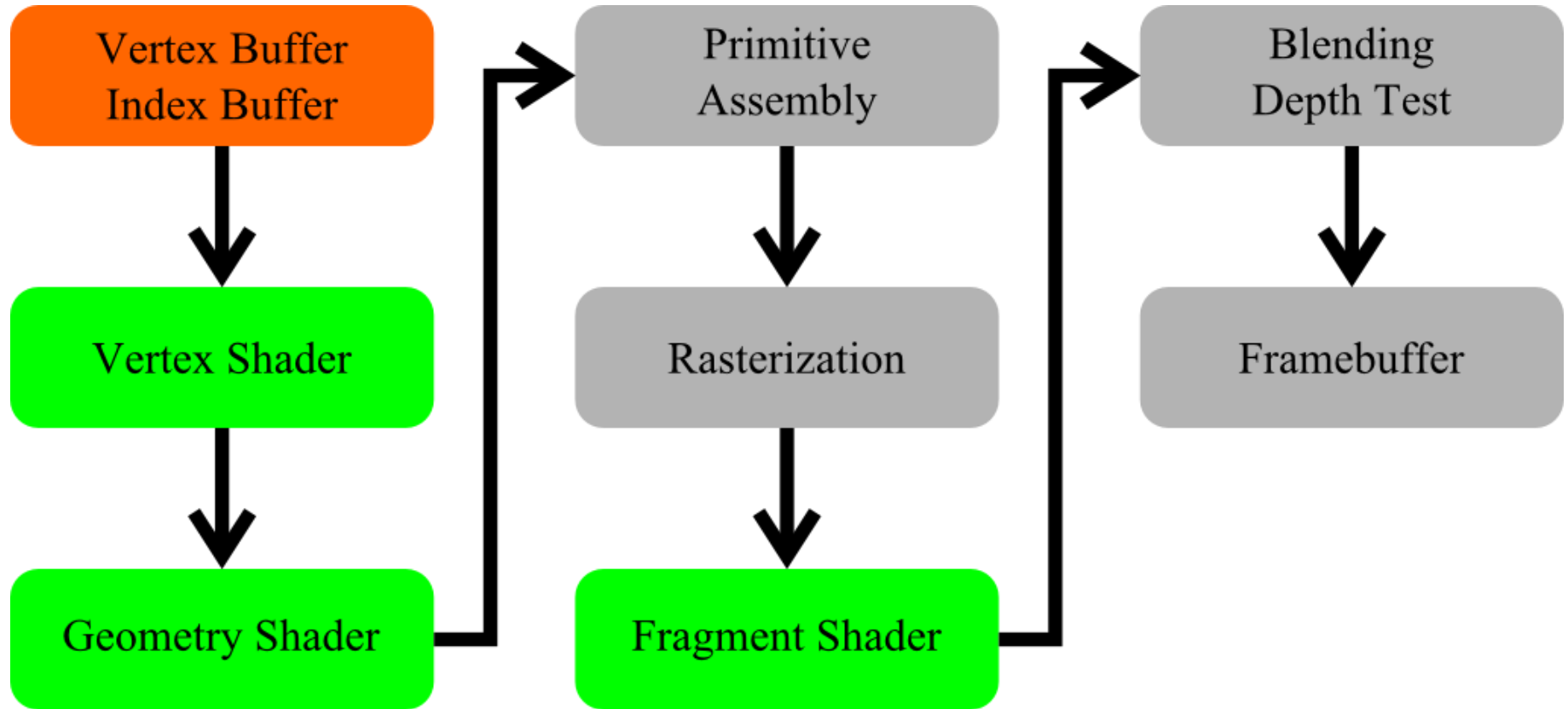
```
canvas = document.getElementById("app");
```

Creating context

```
gl = canvas.getContext("experimental-webgl");  
gl.viewportWidth = 640;  
gl.viewportHeight = 480;  
gl.clearColor(1.0, 0.0, 0.0, 1.0);  
gl.depthFunc(gl.LEQUAL);  
gl.enable(gl.DEPTH_TEST);  
gl.clear(gl.DEPTH_BUFFER_BIT | gl.COLOR_BUFFER_BIT);
```

Output: Red 640x480 rectangle

OpenGL pipeline



Shaders

- Written in OpenGL Shading Language (GLSL)
- C-style language with limitations
- Directly executed on the GPU
- Various types of variables as inputs/outputs

Vertex/Index Buffer

- Vertex buffer contains vertex data
 - Array of 3D positions for vertices
 - Other vertex attributes
 - Texture coordinates
 - Colors
 - Normals
- Index buffer contains triangle information
 - Triplets of indices into the vertex buffer
 - Optional, can use only vertex buffers

Vertex Shader

- Executed once per drawn vertex
- Transforms 3D vertex positions into 2D screen coordinates using projection matrices
 - To determine where to draw the vertices
- Prepares data to be interpolated in a triangle
 - Does not see other vertices
 - Data interpolated based on which triangle the vertex is in
- Receives vertex buffer data

Primitive Assembly / Rasterization

- Primitive assembly connects vertices into triangles using index buffer data
- Individual triangles are sent to the rasterizer
- Rasterization
 - Conversion of triangles (primitives) into pixels
 - Determines which pixels should be filled
 - Based on vertex positions received from vertex shader

Fragment Shader

- Executes once per rasterized pixel
- Receives interpolated data from the vertex shader
- Outputs the final pixel color
- Reads data from textures and applies color
- Calculates lighting (if per-pixel lighting)
 - Per-vertex has lower quality but better performance

Blending/Depth test

- Depth test determines which pixels are in front of which pixels
 - Discards occluded pixels
- Blending determines how the new pixel color and the old pixel color should be mixed
 - Used with transparent objects
- Results are stored to the (back) framebuffer
 - It's the image we will see

WebGL drawing needs

- A vertex and a fragment shader
 - Passed as special script tags
 - Driver compilation and linking
 - Possible build errors
- Geometry
 - Passed as arrays into WebGL vertex buffer objects (VBO) and index buffer objects (IBO)
 - Stored on the GPU after creation
 - One object = one or more VBOs and IBOs

WebGL drawing needs (2)

- Placing the geometry
 - Using a transformation matrix, we place the geometry into a scene
- A camera (just one or two matrices)
 - This gives us our projections that transform vertices in the vertex shader
- Lights
 - If we want lit objects and a natural looking scene

Declaring a vertex shader

```
<script id="sh-vertex" type="x-shader/x-vertex">
attribute vec3 aVertexPosition;// From vertex buffers
attribute vec4 aVertexColor;    // From vertex buffers

uniform mat4 model;              // Model matrix
uniform mat4 viewprojection;    // View-projection matrix

varying vec4 vColor;            // Output to be interpolated

void main() {
    mat4 mvp = model * viewprojection;
    gl_Position = mvp * vec4(aVertexPosition, 1.0);
    vColor = aVertexColor;
}
</script>
```

Declaring a fragment shader

```
<script id="sh-fragment" type="x-shader/x-fragment">  
precision mediump float;  
  
varying vec4 vColor;  
  
void main(void)  
{  
    gl_FragColor = vColor;  
}  
</script>
```


Load shaders into WebGL

```
function loadShader(selector)
{
    var source = $(selector).text();
    var type = $(selector).attr("type");
    var shader;
    if (type == "x-shader/x-vertex")
        shader = gl.createShader(gl.VERTEX_SHADER);
    else if (type == "x-shader/x-fragment")
        shader = gl.createShader(gl.FRAGMENT_SHADER);

    gl.shaderSource(shader, source);
    gl.compileShader(shader);

    if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS))
        return null;
    return shader;
}
```

Create shader program

```
function loadProgram(vertexSelector, fragmentSelector)
{
    var vs = loadShader(vertexSelector);
    var fs = loadShader(fragmentSelector);

    var program = gl.createProgram();
    gl.attachShader(program, vs);
    gl.attachShader(program, fs);
    gl.linkProgram(program);

    if (!gl.getProgramParameter(program, gl.LINK_STATUS))
        return null;

    gl.useProgram(program);
    return program;
}
```

Creating geometry

```
var vertices = new Float32Array([
    -1.0, -1.0, -1.0,
    -1.0, -1.0,  1.0,
    -1.0,  1.0,  1.0,
    -1.0,  1.0, -1.0,
     1.0, -1.0, -1.0,
     1.0, -1.0,  1.0,
     1.0,  1.0,  1.0,
     1.0,  1.0, -1.0,
]);
var indices = new Uint32Array([
    0, 1, 2,    0, 2, 3,    4, 5, 6,    4, 6, 7
]);
```

Creating geometry (2)

```
vbo = gl.createBuffer();  
gl.bindBuffer(gl.ARRAY_BUFFER, vbo);  
gl.bufferData(gl.ARRAY_BUFFER, vertices, gl.STATIC_DRAW);
```

```
vbo2 = gl.createBuffer();  
gl.bindBuffer(gl.ARRAY_BUFFER, vbo2);  
gl.bufferData(gl.ARRAY_BUFFER, vertexColors, gl.STATIC_DRAW);
```

```
ibo = gl.createBuffer();  
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, ibo);  
gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, indices, gl.STATIC_DRAW);
```

Drawing geometry

```
gl.bindBuffer(gl.ARRAY_BUFFER, vbo);  
gl.enableVertexAttribArray(0); // Vertex attrib 0 = Position  
gl.bindAttribLocation(program, 0, "vertexPos");  
gl.vertexAttribPointer(0, 3, gl.FLOAT, false, 0, 0);
```

```
gl.bindBuffer(gl.ARRAY_BUFFER, vbo2);  
gl.enableVertexAttribArray(1); // Vertex attrib 1 = Color  
gl.bindAttribLocation(program, 1, "vertexColor");  
gl.vertexAttribPointer(1, 3, gl.FLOAT, false, 0, 0);
```

```
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, ibo);  
gl.drawElements(gl.TRIANGLES, 12, gl.UNSIGNED_INT, 0);
```

Passing other data to shaders

- Uniform variables
 - Remain constant during a draw call
 - Used for camera, light and material parameters
 - Small amounts of data (few kB)
 - Shared between different types of shaders
- Textures
 - 1D, 2D, 3D arrays of values
 - Accessed in a shader by sampling
 - Potentially unlimited amounts of data

Setting uniform variables

```
var myLocation = gl.getUniformLocation(program, "viewprojection");  
var myLightPos = gl.getUniformLocation(program, "lightPos");
```

```
var myCameraMatrix = new Float32Array([  
    1, 0, 0, 0,  
    0, 1, 0, 0,  
    0, 0, 1, 0,  
    3, 4, 5, 1  
]);
```

```
gl.uniform3f(myLightPos, 1.0, 1.0, -1.0);  
gl.uniformMatrix4fv(myLocation, gl.FALSE, myCameraMatrix);
```

Textures

- Large buffers, usually containing 2D images
 - “wallpapers” that we glue to objects
- Accessed through a sampler object inside a shader
- One shader execution usually accesses only a small area of the texture
- We can even render into textures
 - Shadows, reflections, deferred shading, ...

Textures in WebGL

```
var myTexture;  
function initTexture()  
{  
    myTexture = gl.createTexture();  
    myTexture.image = new Image();  
    myTexture.image.onload = function()  
    {  
        handleLoadedTexture(myTexture);  
    }  
  
    myTexture.image.src = "brickwall.png";  
}
```

Textures in WebGL (2)

```
function handleLoadedTexture(texture)
{
    gl.bindTexture(gl.TEXTURE_2D, texture);
    gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, true);
    gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA,
                  gl.UNSIGNED_BYTE, texture.image);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER,
                     gl.NEAREST);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER,
                     gl.NEAREST);
    gl.bindTexture(gl.TEXTURE_2D, null);
}
```

Textures in WebGL (3)

```
var mySampler = gl.getUniformLocation(program, "texture0");  
var mySampler2 = gl.getUniformLocation(program, "texture1");
```

```
gl.activeTexture(gl.TEXTURE0);  
gl.uniform1i(mySampler, 0);  
gl.bindTexture(gl.TEXTURE_2D, myTexture1);
```

```
gl.activeTexture(gl.TEXTURE1);  
gl.uniform1i(mySampler2, 1);  
gl.bindTexture(gl.TEXTURE_2D, myTexture2);
```

```
gl.DrawElements(...);
```

Shader samplers

```
// Fragment shader
```

```
uniform sampler2D texture0;
```

```
uniform sampler2D texture1;
```

```
varying vec2 textureCoordinate;
```

```
void main()
```

```
{
```

```
    vec4 color = texture2D(texture0, textureCoordinate);
```

```
    vec4 color2 = texture2D(texture1, textureCoordinate);
```

```
    gl_FragColor = (color + color2) / 2.0;
```

```
}
```

Still much work to do

- Load scene from file (such as COLLADA)
 - Geometry, lights, materials, animation sequences
 - Loading similar to texture loading
 - Load models on the fly with AJAX
- Write shaders for lighting and other effects
 - HDR, Deferred Shading, Shadows, Ambient occlusion
 - Motion blur, Depth-of-Field, GPU Skinning
 - ...

Still much work to do (2)

- Handle scene graph for easier scene manipulation
- Handle advanced texture loading
- Physics, various utilities
 - Not directly related to WebGL
- Seems like pure WebGL is too low-level
 - True, but it gives complete control of the pipeline
 - Frameworks might not be that flexible

Frameworks

- **Three.js** <http://mrdoob.github.com/three.js/>
- **PhiloGL** <http://www.senchalabs.org/philogl/>
- **GLGE** <http://www.glge.org/>
- **X3DOM** <http://www.x3dom.org/>
- **CubicVR 3D** <http://www.cubicvr.org/>
- And many others...

Three.js

- Has all the required features
 - Easily create a WebGL renderer
 - Model loaders, cameras, lights, materials, animations
 - Rapid development

Conclusion

- Lots of code, takes a lot of time to code
- High-quality, well-documented frameworks are missing
- Starting from scratch is hard
 - But can provide better performance
 - And you learn a lot
 - Good for old-school pure OpenGL programmers
 - Good opportunity for porting existing games